



**St Francis De Sales College
(Autonomous)**

Department of Computer Science and Applications

V – semester IOT Project Report

Project Title:

Smart Water Monitoring System

Submitted By

Manya V - U03ZW24S0126

Chandana R – U03ZW24S0012

Submitted To

Ms. Maya Mathew

Abstract

The Smart Water Monitoring System is an IoT-based water tank monitoring and pump-control system developed using an ESP32 microcontroller, Wokwi simulation, and the ThingSpeak cloud platform.

The system monitors the water level of a tank using an HC-SR04 ultrasonic sensor. A DHT22 sensor is used to measure temperature and humidity, while a potentiometer is used as a simulated water-quality sensor for the prototype. Based on the sensor readings, the ESP32 controls a water pump through a relay module.

The system operates in automatic and cloud-controlled modes. In automatic mode, the pump is switched ON when the water level falls below 20% and switched OFF when the level rises above 80%. A safety mechanism is implemented so that when the simulated water quality falls below 40%, the pump is switched OFF and an alarm is activated.

The ESP32 communicates with the ThingSpeak cloud platform through Wi-Fi. Sensor data, pump status, alarm status, humidity, and water distance are uploaded to ThingSpeak for remote monitoring. ThingSpeak Field 8 is used to send a pump-control command to the ESP32.

The complete system was designed and tested in the Wokwi simulation environment. Different water-level conditions, cloud commands, and poor-water-quality conditions were tested successfully.

Table of Contents

Sl No	Contents	Pg No
01	Introduction	04
02	Problem Statement	04
03	Objectives	05
04	Components and Software	05
05	System Architecture / Block Diagram	07
06	Circuit Diagram	08
07	Pin Configuration	09
08	Working Principle	10
09	Program Implementation	11
10	ThingSpeak Integration	13
11	Testing and Results	15
12	Applications	20
13	Limitations	20
14	Future Scope	21
15	Learning Outcomes / Reflection	22
16	Conclusion	23
17	Project Links	23
18	References	24
19	Appendix	24

01.Introduction

Water storage tanks are commonly used in homes, buildings, institutions, and other facilities. Conventional water tank management often requires users to manually check the water level and operate the pump. This can result in problems such as insufficient water, unnecessary pump operation, and water overflow.

The Internet of Things provides a method for connecting sensors, controllers, actuators, and cloud platforms to create automated monitoring systems.

The Smart Water Monitoring System developed in this project uses an ESP32 as the main controller. Sensors provide information about the water tank and surrounding environment. The ESP32 processes this information and controls the pump through a relay. The monitored information is also transmitted to ThingSpeak through Wi-Fi.

The project was developed and tested using Wokwi, allowing the complete system to be simulated before considering a physical implementation.

02.Problem Statement

Traditional water tank monitoring systems may require manual observation of water levels and manual control of pumps. Such systems may not provide continuous monitoring or remote access to sensor information.

There is therefore a need for a system that can:

- Monitor water level automatically.
- Monitor temperature and humidity.
- Provide a simulated indication of water quality.
- Automatically control a water pump.
- Generate an alarm when poor water quality is detected.
- Provide visual status indication.
- Send monitoring data to a cloud platform.
- Allow a pump-control command to be received from the cloud.
- Implement safety conditions to prevent unsafe pump operation.

The Smart Water Monitoring System addresses these requirements using an ESP32, sensors, a relay, indicators, Wi-Fi communication, and ThingSpeak.

03.Objectives

The main objectives of the project are:

1. To monitor water level using an HC-SR04 ultrasonic sensor.
 2. To measure temperature and humidity using a DHT22 sensor.
 3. To simulate water-quality measurement using a potentiometer.
 4. To automatically control a water pump using a relay.
 5. To activate an alarm when simulated water quality falls below the defined threshold.
 6. To indicate normal and warning conditions using green and red LEDs.
 7. To transmit sensor data to ThingSpeak.
 8. To receive pump-control commands from ThingSpeak.
 9. To implement safety conditions for poor water quality and high tank level.
 10. To visualize the collected data using ThingSpeak graphs.
-

04. System Components

➤ Hardware Components

- ESP32

The ESP32 acts as the main controller of the system. It receives sensor data, processes the readings, controls the relay and indicators, and communicates with ThingSpeak through Wi-Fi.

- HC-SR04 Ultrasonic Sensor

The HC-SR04 is used to measure the distance between the sensor and the water surface.

The measured distance is used to calculate the water level percentage.

- DHT22 Sensor

The DHT22 measures:

- Temperature
- Relative humidity

- Potentiometer

The potentiometer is used as a simulation of a water-quality sensor.

Its analog reading is converted into a percentage from 0% to 100%.

It should be noted that the potentiometer does not actually measure water quality. It is used only to simulate a water-quality sensor in the Wokwi prototype.

- Relay Module

The relay is connected to the ESP32 and represents the control mechanism for the water pump.

- Green LED

The green LED indicates normal system operation.

- Red LED

The red LED indicates a poor-water-quality warning condition.

- Buzzer

The buzzer provides an audible alarm when poor simulated water quality is detected.

- Resistors

220 Ω resistors are used with the LEDs to limit current.

Software and Platforms

The following software and platforms were used:

- Wokwi — circuit design and ESP32 simulation
- MicroPython — ESP32 programming
- ThingSpeak — cloud data monitoring and control
- GitHub — project documentation and source-code storage

05. Block Diagram

The overall system can be represented as:

Sensors → ESP32 → Actuators

with:

ESP32 ↔ Wi-Fi ↔ ThingSpeak Cloud

The sensors provide measurements to the ESP32. The ESP32 processes the measurements and controls the relay, LEDs, and buzzer. Sensor data and system status are transmitted to ThingSpeak.

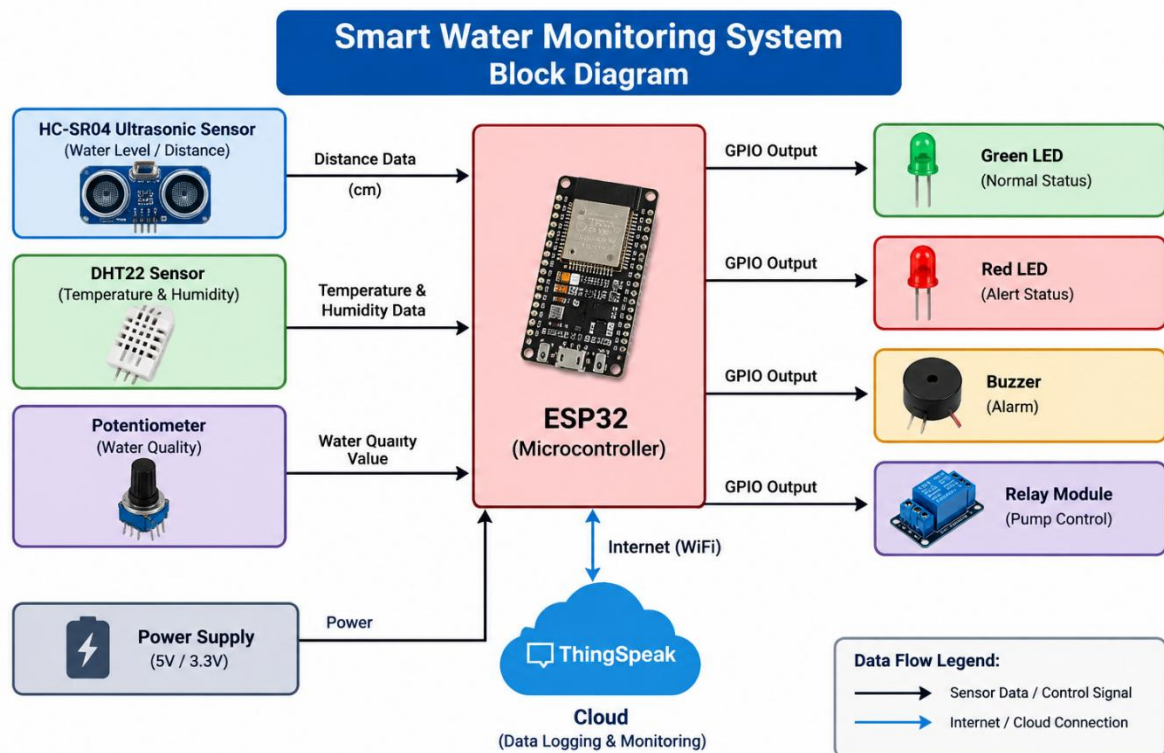


Figure 1: Block Diagram of Smart Water Monitoring System

06. Circuit Diagram

The complete circuit was designed and tested in Wokwi.

The ESP32 is connected to the HC-SR04 ultrasonic sensor, DHT22, potentiometer, relay, LEDs, and buzzer according to the GPIO configuration described above.

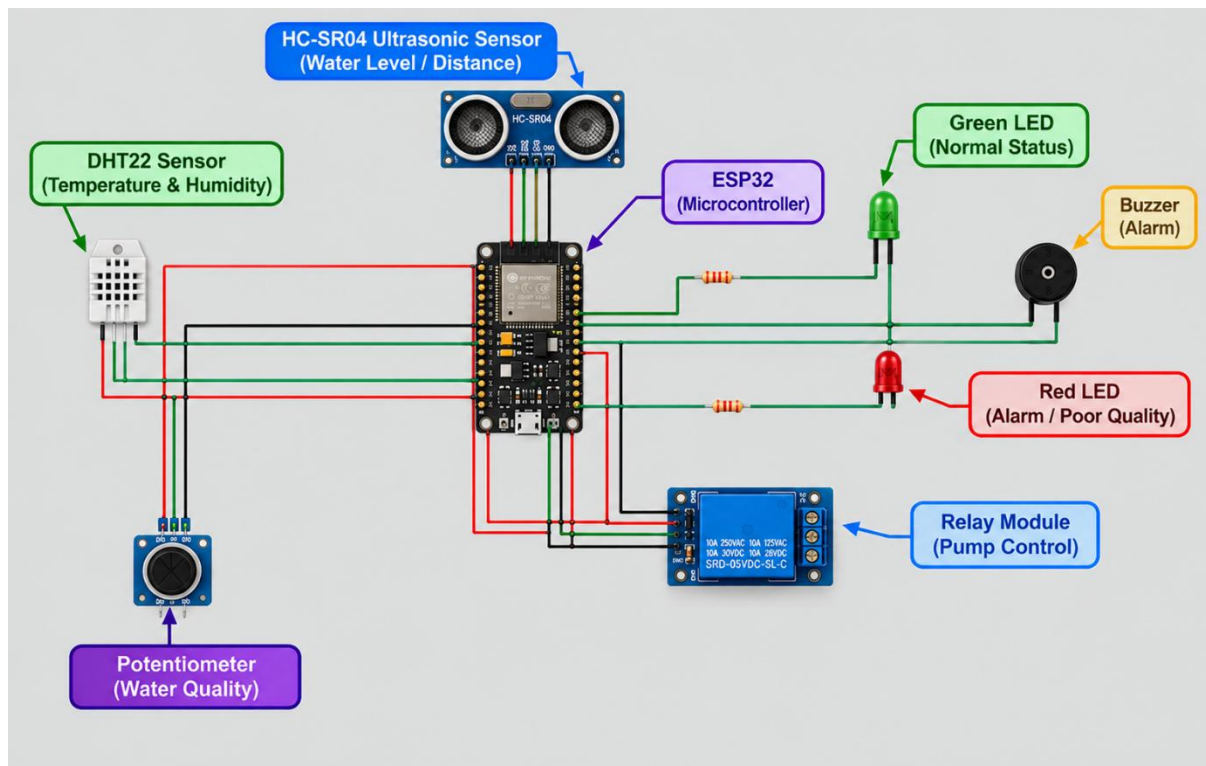


Figure 2: Circuit Diagram of Smart Water Monitoring System

07.Pin Configuration

The final GPIO configuration is shown below.

Component	ESP32 GPIO
DHT22 Data	GPIO 15
HC-SR04 Trigger	GPIO 5
HC-SR04 Echo	GPIO 18
Potentiometer Signal	GPIO 34
Relay IN	GPIO 23
Green LED	GPIO 26
Red LED	GPIO 25
Buzzer	GPIO 27

Power Connections

- DHT22 → 3.3V
- Potentiometer → 3.3V
- HC-SR04 → 5V
- Relay module → 5V
- All components → common GND
- Green LED → GPIO 26 through 220 Ω resistor
- Red LED → GPIO 25 through 220 Ω resistor
- Buzzer → GPIO 27

08. Working Principle

The Smart Water Monitoring System operates continuously by collecting sensor information and processing it using the ESP32.

Step 1: Water-Level Measurement

The HC-SR04 sends an ultrasonic pulse toward the water surface.

The return time of the pulse is measured by the ESP32 and converted into distance.

The water level is calculated using:

$$\text{Water Level (\%)} = ((\text{Tank Height} - \text{Distance}) / \text{Tank Height}) \times 100$$

For this project, the assumed tank height is 100 cm.

The calculated value is restricted between 0% and 100%.

Step 2: Temperature and Humidity Measurement

The DHT22 sensor measures the surrounding temperature and relative humidity.

The ESP32 reads these values and prepares them for display and cloud transmission.

Step 3: Simulated Water-Quality Measurement

The potentiometer produces an analog value between approximately 0 and 4095 through the ESP32 ADC.

The reading is converted into a percentage:

$$\text{Water Quality (\%)} = \text{ADC Value} / 4095 \times 100$$

The project uses the following classification:

Water Quality	Condition
Below 40%	Poor
40% to below 70%	Moderate
70% and above	Good

09. Program Implementation

The ESP32 program was written in MicroPython.

The program performs the following major operations:

1. Import required libraries.
2. Configure GPIO pins.
3. Initialize the DHT22 sensor.
4. Initialize the ultrasonic sensor.
5. Initialize the ADC for the potentiometer.
6. Initialize the relay, LEDs, and buzzer.
7. Connect to Wi-Fi.
8. Read the DHT22.
9. Measure ultrasonic distance.
10. Calculate water level.
11. Read simulated water quality.
12. Read the ThingSpeak Field 8 command.
13. Apply pump-control and safety logic.
14. Update LEDs and buzzer.
15. Upload system information to ThingSpeak.
16. Wait for 20 seconds.
17. Repeat the process.

Important Program Functions

measure_distance()

This function triggers the HC-SR04 sensor and measures the return pulse.

The measured pulse duration is converted into distance in centimetres.

calculate_water_level()

This function converts the measured distance into a water-level percentage based on the assumed 100 cm tank height.

read_pump_command()

This function reads the latest value of ThingSpeak Field 8.

It accepts:

- 0
- 1
- 2

as valid commands.

send_to_thingspeak()

This function uploads the monitored values and system status to ThingSpeak.

The uploaded information includes:

- Water level
- Temperature
- Water quality
- Pump status
- Alarm status
- Humidity
- Distance

10. ThingSpeak Cloud Integration

ThingSpeak is used as the cloud platform for the project.

The ESP32 connects to the Wokwi-GUEST Wi-Fi network and communicates with ThingSpeak using HTTP requests.

The project uses the following ThingSpeak fields:

Field	Parameter
Field 1	Water Level (%)
Field 2	Temperature (°C)
Field 3	Water Quality (%)
Field 4	Pump Status
Field 5	Alarm Status
Field 6	Humidity (%)
Field 7	Water Distance (cm)
Field 8	Pump Control

Fields 1–7 are uploaded by the ESP32.

Field 8 is read by the ESP32 as a cloud command.

This separation prevents the ESP32 from overwriting the pump-control command stored in Field 8.

Pump-Control Logic

The pump is controlled through the relay.

Automatic Mode

When ThingSpeak Field 8 contains command 0, the system operates automatically.

Water Level	Pump Action
Below 20%	Pump ON
20%–80%	Previous state retained
Above 80%	Pump OFF

This provides automatic filling of the tank while preventing operation when the tank is already at a high level.

Cloud Pump-Control Logic

ThingSpeak Field 8 is used as a cloud-control input.

Field 8 Value	Meaning
0	Automatic
1	Force Pump OFF
2	Request Pump ON

When command 1 is received, the pump is switched OFF.

When command 2 is received, the pump is requested to turn ON, provided that the safety conditions permit operation.

Safety Mechanism

A major part of the system is the safety logic.

If the simulated water-quality value falls below **40%**:

- Pump → OFF
- Red LED → ON
- Buzzer → ON
- Alarm Status → ON

This safety condition takes priority over a cloud request to turn the pump ON.

There is also a high-water-level safety condition. If a cloud request is received while the water level is above 80%, the pump remains OFF.

Therefore, cloud control does not bypass the predefined safety conditions.

11. Testing and Validation

The system was tested in multiple stages.

A| Individual Component Testing

The individual components were tested before integration.

Component	Test Result
Potentiometer	ADC value changed when rotated
Green LED	Successfully switched ON
Red LED	Successfully switched ON
Buzzer	Audible alarm produced
Relay	Relay indicator responded
DHT22	Temperature and humidity readings obtained
HC-SR04	Distance readings changed with simulated water position

All individual component tests were successful.

B| Integrated System Testing

After individual testing, the complete system was tested.

Test 1 — Automatic Mode

ThingSpeak Field 8 was set to:

0

The simulated water level was below 20%.

Observed result:

- Water Level $\approx$ 2.7%
- Water Quality > 40%
- Pump $\rightarrow$ ON

- Alarm → OFF
- Relay → ON

Result: Successful

Test 2 — Cloud Force OFF

ThingSpeak Field 8 was set to:

1

Observed result:

- Pump → OFF
- Alarm → OFF
- Relay → OFF

Result: Successful

Test 3 — Cloud Request ON

ThingSpeak Field 8 was set to:

2

The water level was suitable for pumping and water quality was above the safety threshold.

Observed result:

- Pump → ON
- Alarm → OFF
- Relay → ON

Result: Successful

Test 4 — Poor Water-Quality Safety

The potentiometer was adjusted so that the simulated water quality fell below 40%.

Observed result:

- Water Quality $\approx$ 15%
- Pump → OFF
- Alarm → ON
- Red LED → ON

- Buzzer → ON
- Relay → OFF

The poor-water-quality safety condition overrode the pump ON request.

Result: Successful

Test Summary

Test	Input Condition	Expected Result	Observed Result
Automatic	Field 8 = 0, level <20%	Pump ON	Pump ON
Force OFF	Field 8 = 1	Pump OFF	Pump OFF
Request ON	Field 8 = 2, safe condition	Pump ON	Pump ON
Poor quality	Quality <40%	Pump OFF + alarm	Pump OFF + alarm

All four major integrated tests were successfully completed.

ThingSpeak Results

The system successfully transmitted sensor and system information to ThingSpeak.

The ThingSpeak dashboard contains eight configured fields:

1. Water Level
2. Temperature
3. Water Quality
4. Pump Status
5. Alarm Status
6. Humidity
7. Water Distance
8. Pump Control

The graphs provide a visual representation of the values received from the ESP32.

Insert ThingSpeak screenshot here

Smart Water Monitoring System

Channel ID: 3498720

Author: meo000042396411

Access: Private

IoT-based smart water tank monitoring system using ESP32, Wokwi and ThingSpeak. The system monitors water level, temperature, humidity and simulated water quality, and controls a water pump automatically.

iot, esp32, wokwi, smart water, water monitoring, thingspeak

Private View Public View Channel Settings Sharing API Keys Data Import / Export

Add Visualizations Add Widgets Export recent data

MATLAB Analysis MATLAB Visualization

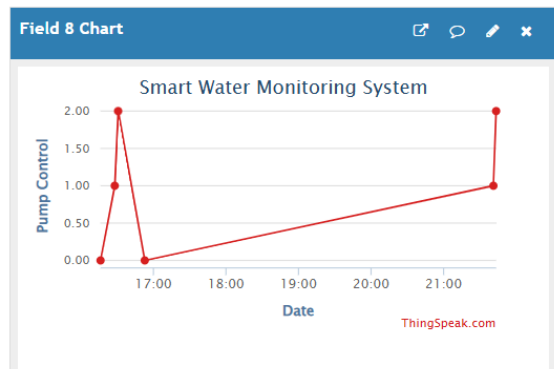
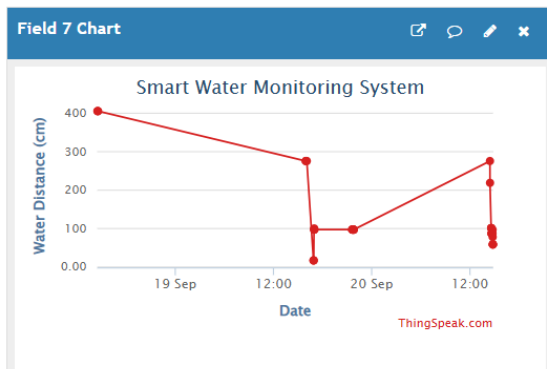
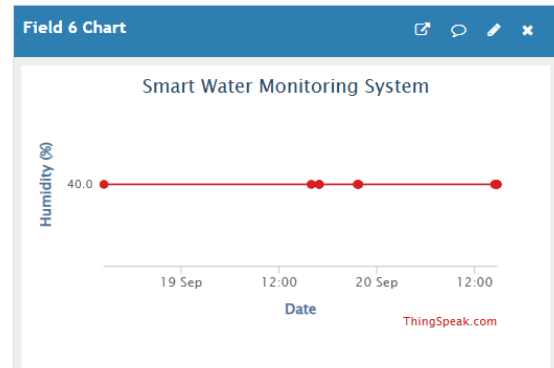
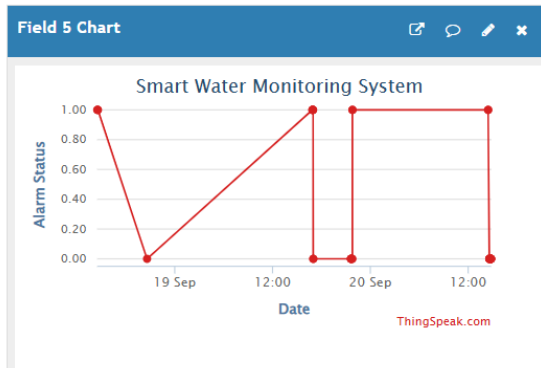
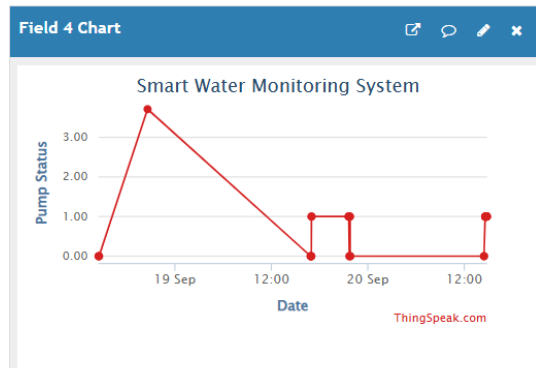
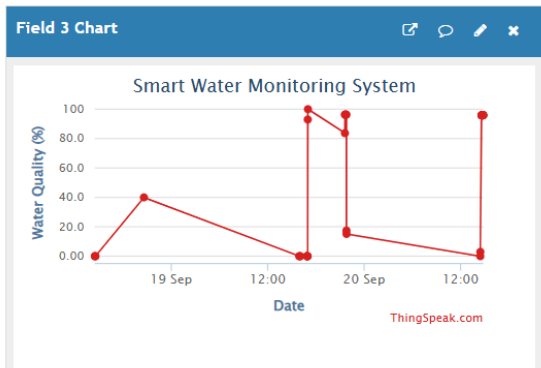
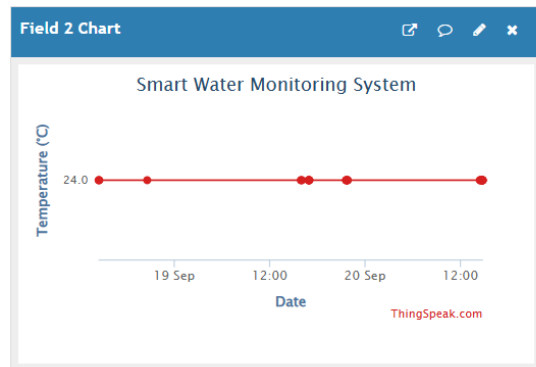
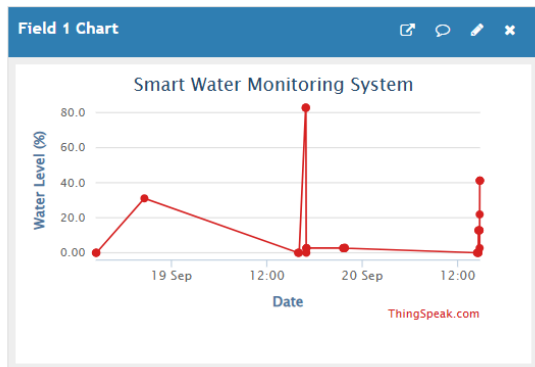
Channel 2 of 2 < >

Channel Stats

Created: 2 days ago
Last entry: 11 minutes ago
Entries: 44



Figure 3: ThingSpeak Dashboard



12.Applications

The system can be applied or adapted for:

- Household water tank management
- Small building water management
- Remote water-level monitoring
- IoT-based water monitoring
- Educational IoT projects
- Automated pump control
- Water-quality alert prototypes

For actual water-quality monitoring, the potentiometer would need to be replaced by an appropriate calibrated water-quality sensor.

13.Limitations

The project has several limitations.

1. The system was developed and tested in Wokwi and has not been tested as a physical hardware installation.
2. The potentiometer is only a simulated water-quality sensor and does not measure actual water-quality parameters.
3. The water-level calculation assumes a tank height of 100 cm.
4. A real HC-SR04-to-ESP32 installation requires appropriate electrical interfacing for the sensor's Echo signal.
5. ThingSpeak communication depends on network availability.
6. The water-level and water-quality thresholds are predefined and may require adjustment for a real installation.
7. The Wokwi relay represents pump control but does not provide the complete electrical protection required for controlling a real water pump.

14.Future Scope

The system can be extended in several ways.

➤ Real Water-Quality Sensor

The potentiometer can be replaced with a suitable calibrated sensor for parameters such as turbidity or other relevant water-quality measurements.

➤ Mobile Application

A mobile application could provide convenient access to sensor readings, pump status, and alarms.

➤ Notifications

Email or mobile notifications could be added for:

- Low water level
- High water level
- Poor water quality
- Pump faults
- Sensor faults

➤ Multiple Tanks

The system could be expanded to monitor and control multiple water tanks.

➤ Data Analysis

Historical ThingSpeak data could be analyzed to identify water-consumption patterns.

➤ Solar Power

A suitable solar-power and battery system could be integrated for applications requiring independent power operation.

➤ Physical Implementation

The Wokwi prototype could be converted into a physical ESP32-based system with appropriate sensor calibration, electrical protection, and a properly rated pump-control circuit.

15. Learning Outcomes / Experiential Learning Reflection

This project provided practical experience in developing an IoT-based monitoring and automation system.

Through the project, we learned how to:

- Design an IoT circuit using ESP32.
- Interface multiple sensors with a microcontroller.
- Read analog sensor values using ADC.
- Measure distance using an ultrasonic sensor.
- Interface a DHT22 sensor.
- Control outputs using GPIO.
- Use PWM to operate a buzzer.
- Control a relay using the ESP32.
- Develop programs using MicroPython.
- Connect an ESP32 to Wi-Fi.
- Communicate with a cloud platform using HTTP.
- Upload sensor data to ThingSpeak.
- Read cloud commands from ThingSpeak.
- Implement automatic control logic.
- Implement safety conditions.
- Test individual components before system integration.
- Debug problems during integration.
- Document the project using GitHub.

The project also helped us understand the complete IoT workflow from

sensor data acquisition → processing → decision making → actuator control → cloud communication → visualization.

16. Conclusion

The **Smart Water Monitoring System** successfully demonstrates an IoT-based approach to water tank monitoring and pump control.

The ESP32 successfully collects information from the HC-SR04, DHT22, and simulated water-quality input. Based on the collected information, the system controls the pump through a relay and provides visual and audible indications through LEDs and a buzzer.

The integration with ThingSpeak enables cloud-based monitoring and remote pump-control commands. The implemented safety conditions ensure that poor simulated water quality and high water level can prevent unsafe pump operation.

The complete system was successfully developed and tested using Wokwi and ThingSpeak. The project demonstrates the practical application of sensors, microcontrollers, actuators, cloud communication, and automation in an IoT system.

17. Project Links

- Wokwi Project

<https://wokwi.com/projects/475514499103966209>

- ThingSpeak Channel

https://thingspeak.mathworks.com/channels/3498720/private_show

- GitHub Repository

<https://github.com/manyagowda1704/Smart-Water-Monitoring-System>

18. References

1. Wokwi — ESP32 simulation and electronics prototyping platform.
 2. ThingSpeak — IoT cloud platform for data collection and visualization.
 3. MicroPython documentation — ESP32 programming and hardware interfaces.
 4. Project-specific circuit design, program code, testing results, and observations developed by the project team.
-

19. Appendix

```
import network
import time
import dht
import urequests

from machine import Pin, ADC, PWM, time_pulse_us

# =====
# THINGSPEAK SETTINGS
# =====

SSID = "Wokwi-GUEST"
PASSWORD = ""
CHANNEL_ID = "3498720"
WRITE_API_KEY = "YOUR_WRITE_API_KEY"
READ_API_KEY = "YOUR_READ_API_KEY"
THINGSPEAK_UPDATE_URL = "https://api.thingspeak.com/update"
THINGSPEAK_FIELD8_URL = (
    "https://api.thingspeak.com/channels/"
    + CHANNEL_ID
    + "/fields/8/last.txt"
```

```

    + "?api_key="
    + READ_API_KEY
)
# =====

# GPIO SETTINGS
# =====

DHT_PIN = 15
TRIG_PIN = 5
ECHO_PIN = 18
QUALITY_PIN = 34
RELAY_PIN = 23
GREEN_LED_PIN = 26
RED_LED_PIN = 25
BUZZER_PIN = 27
# =====

# WATER LEVEL SETTINGS
# =====

TANK_HEIGHT = 100.0
LOW_LEVEL = 20.0
HIGH_LEVEL = 80.0
POOR_QUALITY = 40.0
# =====

# SENSOR SETUP
# =====

dht_sensor = dht.DHT22(Pin(DHT_PIN))
TRIG = Pin(TRIG_PIN, Pin.OUT)
ECHO = Pin(ECHO_PIN, Pin.IN)
quality_sensor = ADC(Pin(QUALITY_PIN))
quality_sensor.atten(ADC.ATTN_11DB)

```

```

# =====

# OUTPUT SETUP

# =====

pump = Pin(RELAY_PIN, Pin.OUT)
green_led = Pin(GREEN_LED_PIN, Pin.OUT)
red_led = Pin(RED_LED_PIN, Pin.OUT)
buzzer = PWM(Pin(BUZZER_PIN))
buzzer.freq(1000)
buzzer.duty_u16(0)

# Start everything OFF

pump.value(0)
green_led.value(0)
red_led.value(0)
buzzer.duty_u16(0)

# =====

# HC-SR04 DISTANCE FUNCTION

# =====

def measure_distance():
    TRIG.value(0)
    time.sleep_us(2)
    TRIG.value(1)
    time.sleep_us(10)
    TRIG.value(0)
    duration = time_pulse_us(ECHO, 1, 30000)
    if duration < 0:
        return None
    return duration / 58.0

# =====

# WATER LEVEL CALCULATION

# =====

```

```

def calculate_water_level(distance):
    if distance is None:
        return None

    level = ((TANK_HEIGHT - distance) / TANK_HEIGHT) * 100.0

    if level < 0:
        level = 0

    if level > 100:
        level = 100

    return level

# =====

# READ THINGSPEAK FIELD 8

# =====

def read_pump_command():
    try:
        response = urequests.get(THINGSPEAK_FIELD8_URL)
        value = response.text.strip()
        response.close()
        command = int(value)
        if command in (0, 1, 2):
            return command

        print("Invalid Field 8 command:", command)
        print("Using automatic mode.")
        return 0
    except Exception as e:
        print("Field 8 read error:", e)
        # Safe fallback to automatic local control
        return 0

# =====

# SEND DATA TO THINGSPEAK

# =====

```

```

def send_to_thingspeak(
    water_level,
    temperature,
    water_quality,
    pump_status,
    alarm_status,
    humidity,
    distance
):
    url = (
        THINGSPEAK_UPDATE_URL
        + "?api_key=" + WRITE_API_KEY
        + "&field1=" + str(round(water_level, 2))
        + "&field2=" + str(round(temperature, 2))
        + "&field3=" + str(round(water_quality, 2))
        + "&field4=" + str(pump_status)
        + "&field5=" + str(alarm_status)
        + "&field6=" + str(round(humidity, 2))
        + "&field7=" + str(round(distance, 2))
    )
    try:
        response = urequests.get(url)
        print("ThingSpeak response:", response.text)
        response.close()
    except Exception as e:
        print("ThingSpeak upload error:", e)

# =====
# WIFI CONNECTION
# =====

```

```

print()
print("=====")
print(" SMART WATER MONITORING SYSTEM")
print(" FULL INTEGRATED TEST")
print("=====")
print()
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
print("Connecting to Wi-Fi...")
wifi.connect(SSID, PASSWORD)
timeout = 20
while not wifi.isconnected() and timeout > 0:
    print("Waiting for Wi-Fi...")
    time.sleep(1)
    timeout -= 1
if not wifi.isconnected():
    print("Wi-Fi connection FAILED.")
    while True:
        time.sleep(5)
print("Wi-Fi connected!")
print("IP address:", wifi.ifconfig()[0])
print()
print("System starting...")
print()
# =====
# MAIN LOOP
# =====
while True:
    print()

```

```

print("-----")
print("READING SYSTEM DATA")
print("-----")
# -----
# DHT22
# -----
try:
    dht_sensor.measure()
    temperature = dht_sensor.temperature()
    humidity = dht_sensor.humidity()
except Exception as e:
    print("DHT22 ERROR:", e)
    temperature = 0
    humidity = 0
# -----
# HC-SR04
# -----
distance = measure_distance()
if distance is None:
    print("HC-SR04 ERROR")
    distance = TANK_HEIGHT
water_level = calculate_water_level(distance)
# -----
# WATER QUALITY
# -----
adc_value = quality_sensor.read()
water_quality = (adc_value / 4095.0) * 100.0
# -----
# THINGSPEAK FIELD 8
# -----

```

```

pump_command = read_pump_command()

# -----

# DISPLAY READINGS

# -----

print()

print("Temperature:", temperature, "C")

print("Humidity:", humidity, "%")

print("Distance:", round(distance, 2), "cm")

print("Water Level:", round(water_level, 2), "%")

print("Water Quality:", round(water_quality, 2), "%")

print("ADC Value:", adc_value)

print()

print("ThingSpeak Pump Command:", pump_command)

# =====

# CONTROL LOGIC

# =====

# -----

# SAFETY PRIORITY:

# POOR WATER QUALITY

# -----

if water_quality < POOR_QUALITY:

    pump.value(0)

    red_led.value(1)

    green_led.value(0)

    buzzer.duty_u16(32768)

    alarm_status = 1

    pump_status = 0

    print()

    print("STATUS: POOR WATER QUALITY")

    print("PUMP: OFF")

```

```

    print("ALARM: ON")
else:
    # Normal water quality
    red_led.value(0)
    green_led.value(1)
    buzzer.duty_u16(0)
    alarm_status = 0

    # =====
    # COMMAND 0 = AUTOMATIC
    # =====

    if pump_command == 0:
        if water_level < LOW_LEVEL:
            pump.value(1)
            print()
            print("MODE: AUTOMATIC")
            print("STATUS: WATER LEVEL LOW")
            print("PUMP: ON")
        elif water_level > HIGH_LEVEL:
            pump.value(0)
            print()
            print("MODE: AUTOMATIC")
            print("STATUS: WATER LEVEL HIGH")
            print("PUMP: OFF")
        else:
            print()
            print("MODE: AUTOMATIC")
            print("STATUS: NORMAL WATER LEVEL")
            print("PUMP: PREVIOUS STATE")

```

```

# =====
# COMMAND 1 = FORCE OFF
# =====

elif pump_command == 1:
    pump.value(0)
    print()
    print("MODE: CLOUD FORCE OFF")
    print("PUMP: OFF")

# =====

# COMMAND 2 = REQUEST ON
# =====

elif pump_command == 2:
    if water_level > HIGH_LEVEL:
        pump.value(0)
        print()
        print("MODE: CLOUD REQUEST ON")
        print("STATUS: TANK LEVEL HIGH")
        print("PUMP: OFF")
        print("HIGH-LEVEL SAFETY ACTIVE")
    else:
        pump.value(1)
        print()
        print("MODE: CLOUD REQUEST ON")
        print("PUMP: ON")
    pump_status = pump.value()
    print("ALARM: OFF")

# =====

# FINAL STATUS
# =====

```

```

print()
print("=====")
print("FINAL STATUS")
print("=====")
print("Pump Status:", pump_status)
print("Alarm Status:", alarm_status)
print("=====")
# =====
# UPLOAD FIELDS 1-7
# =====

print()
print("Sending data to ThingSpeak...")
send_to_thingspeak(
    water_level,
    temperature,
    water_quality,
    pump_status,
    alarm_status,
    humidity,
    distance
)
print()
print("Waiting 20 seconds...")
time.sleep(20)

```